

NotaLang

Un lenguaje de dominio específico para el registro de cursos y el cálculo de notas y promedios ponderados

```
curso Compiladores {  
    PC1 : 15% = 14;    TP : 20% = 16.5;    PC2 : 15%;    TF : 25%;    EB : 25%;  
}  
mostrar(necesito(Compiladores, 13));    // ¿cuánto me falta para llegar a 13?
```

CURSO	Teoría de Compiladores
PROFESOR	José Luis Soncco Álvarez
INTEGRANTES	Alarcón Castellanos, Ericks Santiago — U202322704 Colonio Soto, Rodrigo Aarón — U202323598 Rivera Calderón, Álvaro Jerry — U20231H075
REPOSITORIO	https://github.com/Jere348/NotaLang
VIDEO DEMO	https://youtu.be/acxQxc8Cwto

1. Problemática y motivación

Cada ciclo los estudiantes calculan su promedio en cursos con evaluaciones de distinto peso y necesitan saber cuánto les falta para aprobar. Normalmente esto se hace a mano o en una hoja de cálculo, lo que genera errores frecuentes: pesos que no suman 100 %, notas fuera del rango 0–20 o fórmulas mal copiadas. Una hoja de cálculo no detecta estos problemas porque no conoce las reglas de una evaluación académica.

NotaLang permite declarar cursos y evaluaciones de forma directa, registrar notas y obtener el promedio (`promedio`) o la nota necesaria (`necesito`). El compilador valida las reglas del dominio antes de ejecutar el programa.

2. Objetivos

General: implementar en ANTLR4 los analizadores léxico, sintáctico y semántico de NotaLang.

Específicos:

- Definir el léxico del lenguaje e implementarlo con expresiones regulares.
- Diseñar las gramáticas libres de contexto de cada construcción y validarlas con derivaciones más a la izquierda.
- Implementar la tabla de símbolos y la verificación de tipos y reglas del dominio.
- Probar cada analizador con entradas válidas e inválidas.

3. Construcciones del lenguaje

Tipos de dato: `entero`, `decimal`, `texto`, `logico` y `curso`. Funciones del dominio: `promedio(C)` y `necesito(C, meta)`; `C.E` accede a la nota de la evaluación `E` del curso `C`.

Construcción	Cinco ejemplos
C1 · Declaración de variables	<pre>var nota : decimal = 14.5; var n : entero; var nombre : texto = "Ana"; var aprobado : logico = verdadero; var total : decimal = suma / n;</pre>
C2 · Asignación	<pre>x = 10; prom = suma / n; Compiladores.PC1 = 15; ok = promedio(Calculo) >= 10.5; nombre = "Ana" + " Perez";</pre>
C3 · Expresiones aritméticas, relacionales y lógicas	<pre>a + b * 2 (x - 1) / 2 nota >= 10.5 y no fin promedio(C) < 11 o C.EF == 0 necesito(Calculo, 13) - n mod 2</pre>
C4 · Sentencia selectiva	<pre>si (x > 3) { mostrar(x); } si (ok) { n = 1; } sino { n = 0; } si (p >= 10.5) { mostrar("A"); } sino si (p >= 8) { mostrar("S"); } si (a == b) { } si (no ok) { leer(x); } sino { x = 0; }</pre>
C5 · Sentencias iterativas	<pre>mientras (i < 10) { i = i + 1; } mientras (no ok y ef <= 20) { ef = ef + 1; } para i desde 1 hasta 10 { mostrar(i); } para j desde a hasta b + 1 { s = s + j; } para i desde 1 hasta 3 { leer(nota); suma = suma + nota; }</pre>

C6 · Entrada y salida	<pre> leer(x); leer(Compiladores.EB); mostrar("Hola"); mostrar(a, b, c); mostrar("Necesitas ", necesito(C, 13), " en el EF"); </pre>
C7 · Declaración de curso	<pre> curso Arte { EF : 100%; } curso Calculo { PC : 40% = 12; EF : 60%; } curso Fisica { T1 : 50% = 14; T2 : 50% = 16.5; } curso Quimica { PC1 : 20%; PC2 : 20%; EF : 60% = 11; } curso Ingles { ORAL : 40% = 15.5; ESCRITO : 60%; } </pre>

Tabla 1. Construcciones de NotaLang.

4. Analizador léxico

Tokens con lexema fijo (*token: lexemas*):

Palabras clave	Tipos de dato	Aritméticos y asignación
VAR var	ENTERO entero	MAS · MENOS + -
CURSO curso	DECIMAL decimal	POR · DIV · MOD * / mod
SI si	TEXTO texto	ASIGNA =
SINO sino	LOGICO logico	
MIENTRAS mientras		Delimitadores
PARA para	Literales y operadores lógicos	PAR_IZQ · PAR_DER ()
DESDE desde	VERDADERO verdadero	LLAVE_IZQ · LLAVE_DER { }
HASTA hasta	FALSO falso	DOS_PUNTOS · PUNTO_COMA : ;
LEER leer	Y · O · NO y o no	COMA · PUNTO , .
MOSTRAR mostrar	Operadores relacionales	
PROMEDIO promedio	MENOR · MAYOR < . >	
NECESITO necesito	MENOR_IG · MAYOR_IG <= . >=	
	IGUAL · DISTINTO == . !=	

Tokens definidos con expresiones regulares:

Token	Expresión regular (ANTLR4)	Lexemas de ejemplo
NUM_ENT	[0-9]+	0 , 15 , 2026
NUM_DEC	[0-9]+ '.' [0-9]+	10.5 , 14.25
PORCENTAJE	[0-9]+ '.' [0-9]+ '%'	20% , 12.5%
CADENA	' ~ ["\r\n"]* '	"Hola" , "Promedio: "
ID	[a-zA-Z_] [a-zA-Z_0-9]*	nota , PC1 , Compiladores
(ignorado)	[\t\r\n]+ '//' ~ [\r\n]* '/' * ' * ' /* '	espacios y comentarios (-> skip)

Cualquier otro carácter se reporta como error léxico (**ERROR_LEXICO**) indicando línea y columna.

5. Analizador sintáctico

No-terminales en *cursiva*, terminales en **negrita**; ϵ es la cadena vacía y *programa* el símbolo inicial.

PROGRAMA

programa → *sentencias EOF*
sentencias → *sentencia sentencias* | ϵ
sentencia → *declVar* | *asignacion* | *siStmt* |
mientrasStmt | *paraStmt* | *leerStmt* |
mostrarStmt | *declCurso* | *bloque*
bloque → { *sentencias* }

C1 · DECLARACIÓN

declVar → **var id** : *tipo inicializacion* ;
tipo → **entero** | **decimal** | **texto** | **logico**
inicializacion → = *expr* | ϵ

C2 · ASIGNACIÓN

asignacion → *destino* = *expr* ;
destino → **id** | **id . id**

C3 · EXPRESIONES

expr → *expr o* *exprY* | *exprY*
exprY → *exprY y* *exprNo* | *exprNo*
exprNo → **no** *exprNo* | *exprRel*
exprRel → *exprArit opRel* *exprArit* | *exprArit*
exprArit → *exprArit opSuma* *termino* | *termino*
termino → *termino opMul* *factor* | *factor*
factor → (*expr*) | - *factor* | **num_ent** |
num_dec | **cadena** | **verdadero** |
falso | **id** | **id . id** | **promedio (id)** |
necesito (id , expr)
opRel → < | > | <= | >= | == | !=
opSuma → + | -
opMul → * | / | **mod**

C4 · SELECTIVA

siStmt → **si** (*expr*) *bloque* **sinoParte**
sinoParte → **sino** *bloque* | **sino** *siStmt* | ϵ

C5 · ITERATIVAS

mientrasStmt → **mientras** (*expr*) *bloque*
paraStmt → **para** *id* **desde** *expr* **hasta** *expr* *bloque*

C6 · ENTRADA Y SALIDA

leerStmt → **leer** (*destino*) ;
mostrarStmt → **mostrar** (*argumentos*) ;
argumentos → *expr masArgumentos*
masArgumentos → , *expr masArgumentos* | ϵ

C7 · CURSO

declCurso → **curso** *id* { *evaluaciones* }
evaluaciones → *evaluacion evaluaciones* | *evaluacion*
evaluacion → **id** : **porcentaje** *notaOpcional* ;
notaOpcional → = *expr* | ϵ

Validación mediante derivaciones más a la izquierda

Cada gramática se validó con 4 derivaciones más a la izquierda (36 en total), incluidas en el **Anexo A** (docs/Anexo_Derivaciones.pdf). En C1, C2 y C4–C7, *expr* se abrevia con $\Rightarrow^*$, pues se valida en C3. Ejemplo:

Ejemplo (C7) curso Calculo { PC : 40% = 12; EF : 60%; }

declCurso
 $\Rightarrow_{1m}$ **curso** *id* { *evaluaciones* }
 $\Rightarrow_{1m}$ **curso** *id* { *evaluacion evaluaciones* }
 $\Rightarrow_{1m}$ **curso** *id* { **id** : **porcentaje** *notaOpcional* ; *evaluaciones* }
 $\Rightarrow_{1m}$ **curso** *id* { **id** : **porcentaje** = *expr* ; *evaluaciones* }
 $\Rightarrow_{1m}^*$ **curso** *id* { **id** : **porcentaje** = **num_ent** ; *evaluaciones* }
 $\Rightarrow_{1m}$ **curso** *id* { **id** : **porcentaje** = **num_ent** ; *evaluacion* }
 $\Rightarrow_{1m}$ **curso** *id* { **id** : **porcentaje** = **num_ent** ; **id** : **porcentaje** *notaOpcional* ; }
 $\Rightarrow_{1m}$ **curso** *id* { **id** : **porcentaje** = **num_ent** ; **id** : **porcentaje** ; }

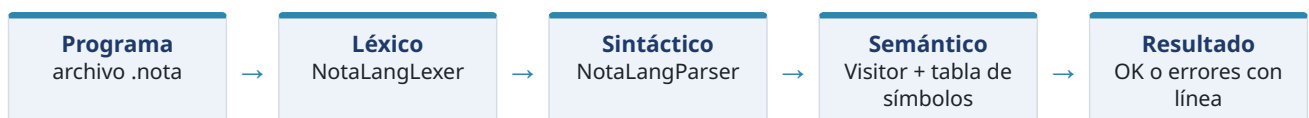
6. Analizador semántico

Se implementó con un *Visitor* de ANTLR y una tabla de símbolos con ámbitos anidados (bloques y para), que guarda nombre, tipo, ámbito y línea de cada identificador. Errores verificados:

Cód.	Error semántico	Ejemplo que lo produce
E1	Identificador (variable o curso) no declarado	<code>b = a + 1;</code> sin declarar <code>b</code>
E2	Identificador ya declarado en el mismo ámbito, o evaluación repetida	<code>var x : entero; var x : decimal;</code>
E3	Tipos incompatibles en asignación (solo se admite entero → decimal)	<code>var edad : entero = 3.5;</code>

E4	Condición de <code>si</code> / <code>mientras</code> que no es lógico	<code>si (n) { ... }</code> con <code>n</code> entero
E5	Los pesos de un curso no suman 100 %	<code>curso Q { PC1 : 30%; EF : 50%; }</code>
E6	Nota fuera del rango [0, 20]	<code>T1 : 50% = 25;</code>
E7	Evaluación inexistente o identificador que no es curso	<code>Ingles.ESCRITO = 15;</code>
E8	Operación con tipos incompatibles	<code>var r : decimal = "a" * 2;</code>

7. Implementación y pruebas



Ejecución: `make` y `python3 main.py archivo.nota [--tokens | --arbol | --tabla]`. `make test` ejecuta los 16 archivos de prueba de `tests/`:

Analizador	Entradas	Resultado
Léxico	<code>ok1-ok3</code> (<code>--tokens</code>), <code>err_lex1</code> , <code>err_lex2</code>	Tokens correctos; detecta <code>#</code> (1:19) y la comilla sin cerrar (1:21)
Sintáctico	<code>ok1-ok3</code> , <code>err_sin1-err_sin3</code>	Acepta los válidos; detecta <code>;</code> faltante, <code>)</code> faltante y curso sin evaluaciones
Semántico	<code>ok1-ok3</code> , <code>err_sem1-err_sem8</code>	Válidos sin errores; cada <code>err_semN</code> reporta el error EN con su línea

Ejemplos de ejecución

```

$ python3 main.py tests/ok3_mientras.nota \
  --tabla
Análisis léxico y sintáctico: OK
=== TABLA DE SÍMBOLOS ===
global Calculo curso línea 1
  evaluaciones={'PC': 40.0, 'EF': 60.0}
global ef entero línea 6
global ok logico línea 7
Análisis semántico: OK
  
```

```

$ python3 main.py tests/err_sem5_pesos.nota
Análisis léxico y sintáctico: OK
=== ERRORES SEMÁNTICOS ===
[Error Semántico E5, línea 1] Los pesos del
curso 'Química' suman 80% (deben sumar 100%)
$ python3 main.py tests/err_sin1.nota
=== ERRORES SINTÁCTICOS ===
[Error Sintáctico, línea 2:0]
missing ';' at 'mostrar'
  
```